Part A: Delivery with A*

(a)

State: A (row, column) tuple which identifies where the delivery robot is on the map.

Initial State: The (row,column) tuple of the building the robot is defined to start at.

Actions: The delivery robot can attempt to move UP, DOWN, LEFT, or RIGHT.

Transition Model: without defining a –slip p value, the transition model is deterministic. The probability of entering a given state s' is entirely influenced by the chosen action a and current state s. If a wall or out of bounds would be moved due to an action, s' = s is given as 100% likelihood. If p is given, each action has a p% chance to fail. Failure means s' = s. This means for the non-deterministic, s' has p% chance to be s, and 100 - p% chance to be determined by s and a.

Goal Test: Each arrival after an action checks whether the delivery robot is at the defined destination building. If it has reached the destination for the first time, it has reached its goal.

Path Cost: Reward per action is either -1 when goal test is false, or 39 if goal test is true. The total path cost is 40 - L, where L is the number of actions taken.

why BFS is optimal: Breadth First Search (BFS) is optimal for fixed-cost maps because breadth first search methods always explore states in ascending order of distance from start. This always yields the shortest path as output, as if the output path wasn't the shortest, the search would need to explore states that were a longer distance away from start than the optimal path state first. Shortest paths in fixed-cost maps are also the most rewarding/cost efficient.

(b)

why manhattan distance: A* requires a heuristic estimate h(n), which returns an estimated cost of going from n to goal. Manhattan distance is a type of heuristic you can use to output this estimated cost, as it scales its output based on the total height and length to go from n to goal.

(c)

A* vs BFS route length: In the table below, A* always finds a route with length equal to the shortest route. However, A* may sometimes not output the shortest route, as depending on what g function you use as your heuristic, it may incorrectly predict what path is closest to the goal.

A* vs BFS expanded node count: A* usually expands fewer nodes than BFS. However, when the g heuristic function is bad, or gives you wrong predictions for a given state, A* can expand more nodes than BFS.

Map	Delivery Task	BFS Route Length	A* Route Length	BFS Expanded Node Count	A* Expanded Node Count
Baseline	S->W	18	18	67	22
Baseline	L->D	22	22	79	62
Baseline	H->P	16	16	41	16
Detour	S->W	26	26	70	51
Detour	L->D	22	22	65	60
Detour	H->P	16	16	41	16

Part B: Learn a delivery policy

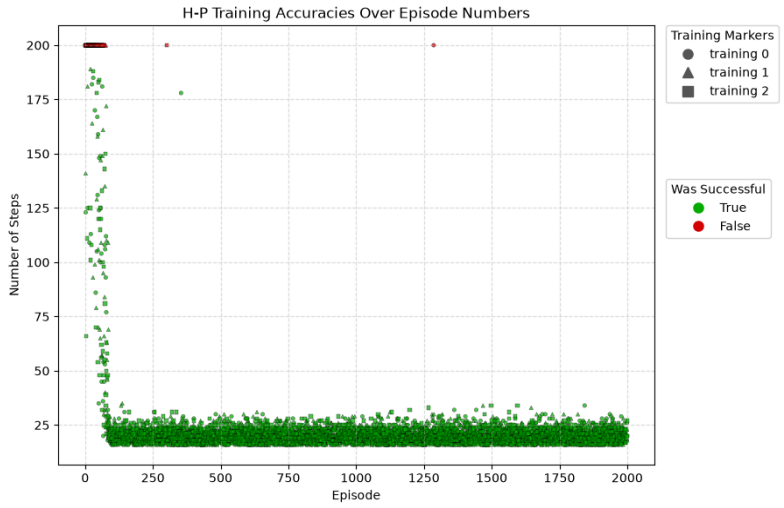
(a)

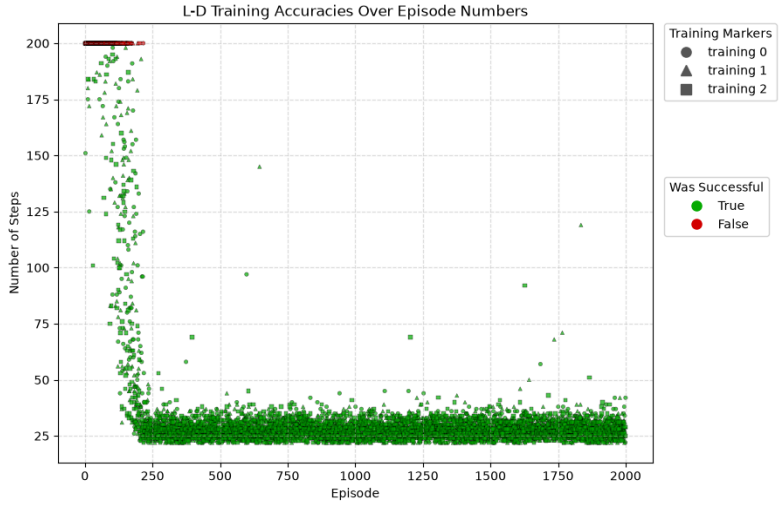
All three Start->Goal pairs' three separate random seeded models return success rates of 1. This means all models trained were 100% accurate in finding a valid path from Start to Goal in eval period (when $\varepsilon = 0.0$).

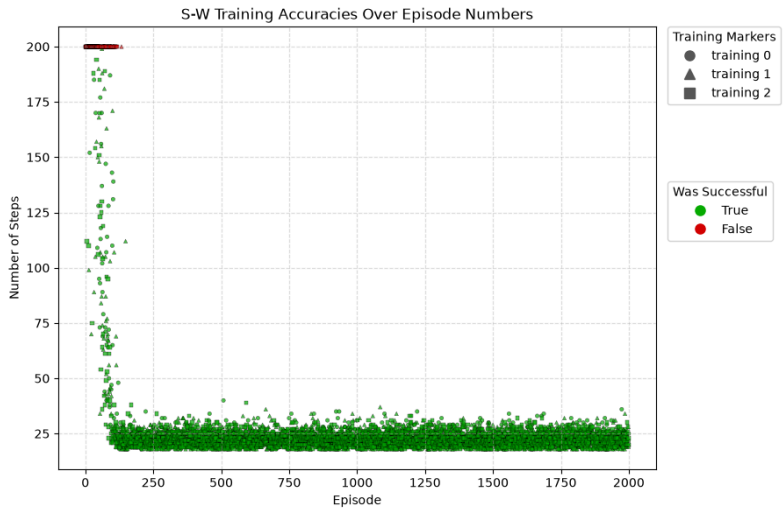
Furthermore, all evaluation episodes ran on the models used the minimum steps found in their respective training episodes. In fact, each triplet of models trained on the same Start->Goal pair converged onto a minimum step count in their training data, which was the step count seen on all evaluation episodes. These results make sense, as if $\varepsilon = 0.0$, the model will always choose the action that maximizes the q value of the next state.

(b)

Graphs of Training Runs for S->W, L->D, and H->P:







Part C: Design a map

(a)

my_campus.txt:

```
##################
#S........########
#.##.####....L...#
#.##.####.####.#.#
#.##.#P##........#
#.##.######.######
#.##...D###.######
#.#H.######.....W#
##################
```

BFS:

Expanded: 51
Path Length: 21

A*:

Expanded: 50
Path Length: 21

(c)

(i)

How do independent movement failures affect final evaluation step variance when comparing separately seeded runs?

I believe that as slip p increases, the variance of the evaluation steps will increase. I believe this because slip p will add randomness to the actions actual effect to the state of the robot, causing some actions to behave unpredictably. As more unpredictable states are reached, the robot will inevitably reach a state that isn't on the optimal path, leading to backtracking and ultimately more steps than needed for some episodes.

(ii)

My custom map is very wall heavy, and has multiple long hallways that the robot may get lost in during training. There is a loop to the bottom right of L and a dead end directly below S. There are three routes the robot could choose to take from S->W. One is the vertical hallway that bypasses going through L, one is +1 x from L, and the last is +3 x from L.

(iii)

Experiment conduct: I kept all factors except for slip p on their default values, and input maps/my_campus.txt as the supplied map. I changed p from 0.0, 0.1, then 0.2 for three separate runs of robot training.

Experiment Commands:
python3 run_project.py train --map maps/my_campus.txt --slip 0.0 --start S --goal W --output results/custom_p00

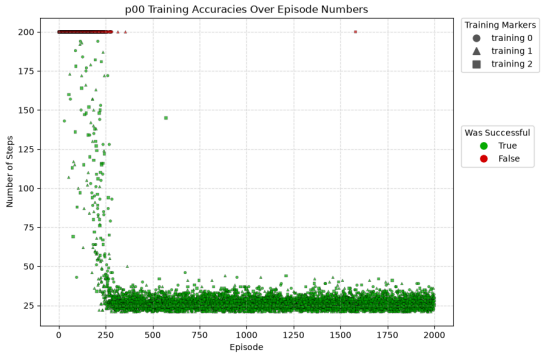
python3 run_project.py train --map maps/my_campus.txt --slip 0.1 --start S --goal W --output results/custom_p01

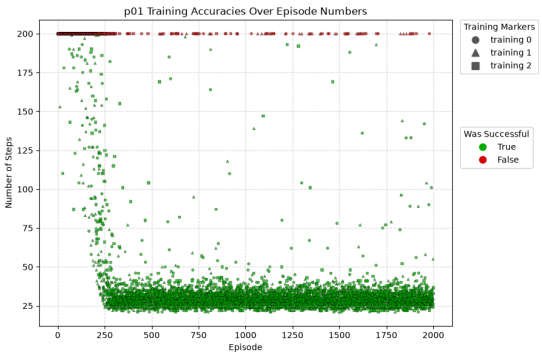
python3 run_project.py train --map maps/my_campus.txt --slip 0.2 --start S --goal W --output results/custom_p02

python3 summarize_results.py results/custom_p00
python3 summarize_results.py results/custom_p01
python3 summarize_results.py results/custom_p02

(iv)

slip number	p = 0.0	p = 0.1	p = 0.2
variance of mean step to success per seed	0	0.0089	5.4134







(v)

These results align with my hypothesis: as slip p increases, variance between different seeded evaluation trials also increases. An interesting outcome is the success rate of all three training seeds when p=0.1 was 1.0. I think this is statistically unlikely because this means the random change of incorrectly moving (adding randomness to actions) didn't once in the 300 trials of evaluation make the robot not complete the task in under 200 actions. I believe if I re-tested this specific p value, I wouldn't recreate this result.